COMPLETE ANALYSIS EXECUTION PLAN

Cross-Regional Transferability of EuroSAT-Pretrained ResNet-50 for LULC Classification in Lahore, Pakistan

Prepared For	Muhammad Shahid Usman — MS Data Science
University	Riphah International University
Project Duration	April 2026 – October 2026 (6 Months)
Hardware	Local: Intel i7-8850H · 16GB RAM · Quadro P2000 4GB + Google Colab T4 GPU
Document Purpose	Step-by-step practical guide covering every task from environment setup to journal submission

Phase	Focus Area	Duration	Platform
Phase 0	Environment & Account Setup	1 week	Local Machine
Phase 1	EuroSAT Pretraining — Source Domain	3 weeks	Google Colab T4
Phase 2	Lahore Sentinel-2 Data Acquisition	2 weeks	Google Earth Engine
Phase 3	Ground Truth Labeling	4 weeks	QGIS + Field
Phase 4	Baseline, Model A & Model B Training	4 weeks	Colab T4 + Local
Phase 5	Sample Efficiency Experiment	2 weeks	Local / Colab
Phase 6	Domain Shift Analysis — t-SNE	2 weeks	Local Machine
Phase 7	Results, Figures & Paper Writing	4 weeks	Local Machine
Phase 8	Peer Review & Journal Submission	3 weeks	MDPI Online

PHASE 0 Environment & Account Setup

Before writing a single line of project code, your accounts, tools, and folder structure must all be in place. Skipping this phase causes messy, hard-to-reproduce work later.

0.1 Accounts to Create or Verify

- **Google Account:** Required for Google Earth Engine, Google Colab, and Google Drive. One account covers all three.
- **Google Earth Engine (GEE):** Go to earthengine.google.com and register for a non-commercial research account. Approval usually takes 1–2 days.
- **Google Colab:** Accessible automatically with your Google Account at colab.research.google.com. Select T4 GPU as your runtime (Runtime → Change runtime type → T4 GPU).
- **GitHub:** Create a repository named eurosat-lahore-lulc. Set it to Public for open-science credibility. Commit after every working session.
- **Kaggle:** Optional backup GPU. Register at kaggle.com — gives you 30 hours of free P100/T4 GPU per week.
- **Zenodo:** Register at zenodo.org using your ORCID or GitHub. This is where your labeled Lahore dataset will be published.

0.2 Local Software Installation

1. **Python 3.10+:** Install via Anaconda (recommended) or python.org. Anaconda makes environment management much easier.
2. **QGIS 3.x:** Download from qgis.org. Free and open source. This is your primary ground truth labeling and map-making tool.
3. **ArcGIS Pro 3.x:** Use your student license from the university. Required only for final cartographic map outputs.
4. **VS Code or PyCharm:** Your code editor. VS Code is lighter and works well with Jupyter notebooks locally.
5. **Git:** Install from git-scm.com. Used to push your work to GitHub after each session.
6. **Google Earth Pro:** Download free from earth.google.com. Used as high-resolution reference imagery during ground truth labeling.

0.3 Python Environment Setup

Create a dedicated conda environment to keep your project dependencies isolated. Open Anaconda Prompt or terminal and run the following commands in sequence:

Commands to Run in Your Terminal

conda create -n lulc python=3.10 → conda activate lulc → pip install torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu118 → pip install rasterio geopandas scikit-learn matplotlib seaborn pandas numpy tqdm jupyter

- **Verify GPU:** Open Python and run: import torch; print(torch.cuda.is_available()); print(torch.cuda.get_device_name(0)). It must print True and NVIDIA Quadro P2000.
- **If it prints False:** Run nvidia-smi in your terminal to see your CUDA driver version, then install the matching PyTorch CUDA build from pytorch.org.

0.4 Project Folder Structure

Create this exact folder structure on your local machine at the start. Consistent structure prevents confusion across eight phases of work.

Recommended Folder Structure

```
eurosat-lahore-lulc/
├── data/
│   ├── eurosat_ms/          ← Raw EuroSAT GeoTIFF patches
│   ├── lahore_sentinel2/    ← Exported Lahore GeoTIFF from GEE
│   ├── lahore_patches/      ← 64×64 tiled patches
│   └── ground_truth/        ← Your CSV labels + shapefiles
├── models/                  ← Saved .pth checkpoint files
├── notebooks/               ← Jupyter notebooks per phase
├── results/                 ← Confusion matrices, metrics CSVs
├── figures/                 ← All publication-ready figures
└── paper/                   ← Manuscript drafts
```

PHASE 1 EuroSAT Pretraining — Source Domain

This phase trains your ResNet-50 on EuroSAT Multispectral data. The saved checkpoint from this phase is the starting point for both Model A (zero-shot) and Model B (fine-tuned). Run this entirely on Google Colab T4 GPU.

PHASE 1 EuroSAT Pretraining

Duration: 3 Weeks · Target: May 2026, Weeks 1–3

Step 1.1 — Download EuroSAT Dataset

#	Action	Tool / Where	Expected Output	Success Check
1	Go to Zenodo and search for EuroSAT MS (DOI: 10.5281/zenodo.7711810). Download the EuroSAT_MS.zip file (approx. 2.8 GB).	Zenodo website (zenodo.org)	EuroSAT_MS.zip file on your local machine	File size ~2.8 GB. Unzip successfully into 10 class folders.
2	Unzip the archive. You will see 10 folders, one per class (AnnualCrop, Forest, HerbaceousVegetation, Highway, Industrial, Pasture, PermanentCrop, Residential, River, SeaLake), each containing GeoTIFF files.	File Explorer or unzip command	10 class folders with .tif patch files inside each	Each folder has 2,000–3,000 .tif files. Total ~27,000 patches.
3	Upload the entire EuroSAT_MS folder to your Google Drive under a folder called /project/data/eurosat_ms. This makes it accessible in Google Colab.	Google Drive (drive.google.com)	EuroSAT data accessible from Colab via drive.mount()	Browse to the folder in Drive and confirm all 10 subfolders are present.

Step 1.2 — Compute Dataset Statistics

Before training you must know the mean and standard deviation of each of the 13 Sentinel-2 bands across the entire EuroSAT training set. These values are used to normalize every patch during training and testing. Without correct normalization your model will not converge properly.

What to Compute

Open a Colab notebook. Loop through all training patches, load each as a 13-band array, accumulate the running sum and sum-of-squares per band, then compute mean = sum/N and std = sqrt(sum_sq/N - mean²). Save these 26 values (13 means + 13 stds) to a JSON file called eurosat_stats.json and download it to your local models/ folder.

Step 1.3 — Build the Dataset Loader

Write a PyTorch Dataset class that: (1) reads a GeoTIFF patch using Rasterio, (2) returns all 13 bands as a float32 tensor, (3) applies per-band normalization using the statistics you computed, and (4) applies augmentations (random horizontal flip, vertical flip, rotation up to 15°, brightness jitter 10%) to the training split only. The test split must receive normalization only — no augmentation ever.

Step 1.4 — Modify ResNet-50 Architecture

Download the standard ResNet-50 with ImageNet pretrained weights from torchvision. Change exactly one thing: replace the first convolution layer (originally expecting 3 RGB channels) with a new convolution layer that expects 13 channels. Keep the kernel size (7×7), stride (2), and padding (3) identical. Initialize the new 13-channel weights by averaging the original 3-channel weights across the input dimension — this preserves useful low-level feature detectors. Replace the final fully connected layer with a new linear layer outputting 10 classes (the 10 EuroSAT classes).

Step 1.5 — Train on EuroSAT

Hyperparameter	Value & Reason
Optimizer	AdamW — better weight decay handling than Adam for fine-tuning tasks
Learning Rate	1×10^{-4} initial — standard for fine-tuning a pretrained backbone
LR Schedule	Cosine annealing — smoothly decays LR to near zero by final epoch
Batch Size	16 — use 16 (not 32) to stay within Colab T4's 16GB VRAM safely
Epochs	50 maximum with early stopping patience of 5 epochs
Loss Function	Cross-entropy with label smoothing $\varepsilon=0.1$ — reduces overconfidence
Early Stopping	Monitor validation accuracy. Stop if no improvement for 5 epochs.
Checkpoint Saving	Save only the best model (highest val accuracy) as eurosat_resnet50.pth

Step 1.6 — Evaluate on EuroSAT Test Set

After training, load the best saved checkpoint and evaluate it on the EuroSAT test split (5,400 patches). Record Overall Accuracy, per-class F1-score, Cohen's Kappa, and save the full confusion matrix.

Success Criterion — Do Not Proceed Until Met

Your EuroSAT test accuracy must be at least 93% Overall Accuracy. If it is below this threshold, something is wrong — check your normalization statistics, your 13-channel weight initialization, and whether augmentation was accidentally applied to the test set. A weak EuroSAT model invalidates all downstream comparisons.

Step 1.7 — Download and Archive the Checkpoint

- **Download eurosat_resnet50.pth:** from Colab to your local models/ folder. This file is your single most important output from Phase 1.
- **Also download eurosat_stats.json:** (the 13-band mean/std values). You will need these every time you load any Lahore patch.
- **Commit to GitHub:** Commit your training notebook and eurosat_stats.json. Do not commit the .pth file (too large) — instead upload it to Google Drive and note the link in your README.

PHASE 2 Lahore Sentinel-2 Data Acquisition

This phase acquires your target domain data — Sentinel-2 imagery covering all of Lahore District for 2023 — and tiles it into 64×64 pixel patches matching the EuroSAT format. All acquisition work happens in Google Earth Engine (GEE) through your browser. No GPU needed.

PHASE 2 Lahore Data Acquisition

Duration: 2 Weeks · Target: May 2026, Weeks 4–5

Step 2.1 — Lahore District Boundary

- **Download GADM v4.1 boundary:** Go to gadm.org and download Pakistan's district-level shapefile. Open in QGIS, filter to Lahore District, and export as a new shapefile called lahore_district.shp.
- **Upload to GEE as an Asset:** In the GEE Code Editor, go to Assets → New → Shape files. Upload your lahore_district.shp (plus the accompanying .dbf, .shx, .prj files). Name the asset lahore_boundary. Wait for ingestion to complete (5–15 minutes).

Step 2.2 — Build the Sentinel-2 Composite in GEE

In the GEE Code Editor (code.earthengine.google.com), write and execute a script with the following logic:

7. **Load your asset:** Use ee.FeatureCollection with your lahore_boundary asset path to define the study area geometry.
8. **Filter the image collection:** Load COPERNICUS/S2_SR_HARMONIZED. Filter by date to October 1 – November 30, 2023. Filter by the Lahore boundary. Filter to cloud cover below 10% using the CLOUDY_PIXEL_PERCENTAGE property.
9. **Apply cloud masking:** Use the QA60 band to mask cloud and cirrus pixels from individual images before compositing.
10. **Select your 10 bands:** Select B2, B3, B4, B5, B6, B7, B8, B8A, B11, B12. Do not include B1, B9, B10 (these are atmosphere/water vapour bands at coarser resolution or lower utility).
11. **Create a median composite:** Apply .median() to the filtered, cloud-masked collection. This produces a single cloud-free image representing the typical reflectance over October–November 2023.
12. **Clip to boundary:** Apply .clip(lahoreGeometry) to restrict the output to Lahore District only.

Step 2.3 — Export from GEE to Google Drive

Use Export.image.toDrive() in your GEE script with these exact settings: scale = 10 (metres per pixel), region = your Lahore geometry, crs = EPSG:32643 (UTM Zone 43N — the correct projection for Lahore), maxPixels = 1e13, fileFormat = GeoTIFF, folder = 'lahore_s2'. Click Run in the Tasks panel to start the export. The export will take 15–40 minutes. Monitor progress in the Tasks panel.

If Cloud Cover Is a Problem

If too many pixels are masked and you get a patchy composite, extend your date range to September 1 – December 31, 2023. The post-monsoon window provides excellent clear-sky conditions for Lahore. Alternatively, try a minimum-NDWI composite strategy instead of median to prioritize drier, clearer pixels.

Step 2.4 — Download and Inspect the GeoTIFF

- **Download from Google Drive:** to your local lahore_sentinel2/ folder.
- **Open in QGIS:** Load the GeoTIFF. Display as a false-color composite (Band 4 = Red, Band 3 = Green, Band 2 = Blue for natural color; or Band 8, Band 4, Band 3 for NIR false-color). Verify you can visually identify built-up areas, agricultural land, the Ravi River, and open fields.
- **Check for data gaps:** Look for any large black or NoData areas. If gaps exist, they indicate cloud-contaminated or missing tiles. Extend your date range and re-export.

Step 2.5 — Tile into 64×64 Patches

Write a Python script using Rasterio that systematically tiles your Lahore GeoTIFF into non-overlapping 64×64 pixel patches. For each patch, save: (1) the 10-band GeoTIFF patch file named by a unique ID (e.g., lahore_patch_0001.tif), and (2) a metadata CSV row recording the patch ID, top-left pixel coordinates in EPSG:32643, and the corresponding latitude/longitude of the patch center in WGS84.

- **Discard water-only patches:** Patches that are entirely water or entirely NoData can be excluded from labeling.
- **Expected patch count:** Lahore District at 10m resolution covers roughly 17,720,000 pixels. You will generate approximately 4,000–8,000 non-overlapping 64×64 patches. You only need to label about 800–1,300 of them for ground truth.

PHASE 3 Ground Truth Labeling

This is the most time-intensive phase and the one most likely to cause delays if underestimated. You are manually assigning a land cover class label to individual 64×64 pixel Sentinel-2 patches. The quality of your labels directly determines the reliability of every result in this study. Work carefully and consistently.

PHASE 3 Ground Truth Labeling

Duration: 4 Weeks · Target: June 2026

Step 3.1 — Define Your Class Schema

Your Lahore classes must be defined before labeling begins and must not change mid-way through. Align them to EuroSAT's 10 classes where possible, but adapt for Lahore's landscape reality. Recommended class schema:

Lahore Class	EuroSAT Equivalent	Spectral Appearance in Lahore
Residential / Settlements	Residential	Dense grey/brown irregular texture; informal housing dominant
Agriculture — Irrigated	Annual Crop / Permanent Crop	Green to light brown; regular small field patterns
Forest / Plantation	Forest	Dark green; mostly along canal banks and parks
Barren / Exposed Soil	Pasture (partial)	Bright beige or tan; open dry land with no vegetation
Water Bodies	River / Sea Lake	Dark blue or black; Ravi River and irrigation canals
Industrial / Commercial	Industrial	Large grey uniform structures; peri-urban
Roads / Highway	Highway	Linear narrow grey features; visible in 10m imagery

Step 3.2 — QGIS Labeling Workspace Setup

13. **Load the Lahore composite:** Open your lahore_sentinel2.tif in QGIS as a false-color composite.
14. **Add Google Satellite as basemap:** In QGIS, go to XYZ Tiles → right-click → New Connection. URL: https://mt1.google.com/vt/lyrs=s&x={x}&y={y}&z={z}. This gives you high-resolution visual reference imagery directly inside QGIS.
15. **Load patch centroids:** Import your patch metadata CSV as a Delimited Text Layer using the latitude/longitude columns. You will see all patch centers as point features on the map.
16. **Create labeling layer:** Create a new GeoPackage point layer with fields: patch_id (text), class_label (text), confidence (integer, 1–3), source (text: visual or field), notes (text).

17. **Save frequently:** Every 20–30 labels, save your QGIS project. Labels are your most irreplaceable output.

Step 3.3 — Labeling Protocol

- **Target sample counts:** Aim for minimum 130 labeled patches per class — 100 for training and 30 held out for testing. With 7 classes, this means approximately 910 total labeled patches.
- **Confidence scoring:** Assign confidence 3 (certain), 2 (likely), or 1 (uncertain). Only confidence 3 and 2 samples should be used in training. Confidence 1 samples go into a review pile.
- **Spatial spread:** Do not label 100 samples from the same neighborhood. Spread your samples across different parts of Lahore District to capture within-class spectral variability.
- **Train/test split now:** Randomly assign 100 samples per class to training and 30 to testing before you begin any model training. The test set must never be touched until final evaluation.

Step 3.4 — Field Verification

Plan 2–3 field days in Lahore targeting ambiguous classes — particularly informal settlements at the urban fringe, mixed agricultural areas near Sheikhupura Road, and areas where barren land and sparse vegetation are hard to distinguish from satellite imagery. During field visits:

- Carry your patch centroids loaded on Google Maps or Maps.me on your phone for navigation
- Take GPS-tagged photographs at each visited location using your phone camera
- Record: GPS coordinates, observed land cover class, dominant land use, date and time
- On returning, update the confidence scores and correct any misclassified labels in your QGIS layer

Step 3.5 — Final Label Export

- **Export to CSV:** From QGIS, export your labeling layer as ground_truth_final.csv with columns: patch_id, class_label, class_id (integer 0–6), confidence, source, split (train or test).
- **Generate a summary table:** Count samples per class per split. Confirm no class has fewer than 30 test samples.
- **Commit to GitHub:** Push the CSV to your repository. This is a key project deliverable.

CRITICAL: Never look at test set patches during labeling analysis

Once you assign a patch to the test split, do not examine its predictions or use it to make decisions about your labeling strategy. The test set must remain completely blind until your three models are fully trained. Peeking at test predictions and adjusting labels based on model errors invalidates your results.

PHASE 4 Model Training — Baseline, Model A & Model B

You now have everything you need: a pretrained EuroSAT checkpoint (Phase 1), Lahore patches (Phase 2), and ground truth labels (Phase 3). This phase trains and evaluates all three experimental models under identical conditions.

PHASE 4 Three-Model Training & Evaluation

Duration: 4 Weeks · Target: July 2026

Step 4.1 — Lahore Dataset Loader

Write a second PyTorch Dataset class specifically for Lahore patches. It must use the same normalization statistics (eurosat_stats.json) that you used for EuroSAT — this is essential for fair comparison, because all three models must receive identically preprocessed inputs. Apply the same training augmentations as Phase 1 (random flips, rotation, brightness jitter). Test split receives normalization only.

Step 4.2 — Baseline Model (ImageNet → Fine-tuned on Lahore)

18. **Load:** Standard ResNet-50 with ImageNet pretrained weights from torchvision.models.resnet50(pretrained=True).
19. **Modify:** Replace only the final fully connected layer with a new linear layer matching your number of Lahore classes.
20. **Train:** Fine-tune all layers on Lahore training set. AdamW optimizer, learning rate 1×10^{-4}, cosine annealing, 30 epochs, batch size 16, early stopping patience 5.
21. **Save:** Save best checkpoint as baseline_lahore.pth.
22. **Evaluate:** Run on the Lahore test set. Record OA, per-class F1, Kappa, and confusion matrix.

Step 4.3 — Model A (EuroSAT Pretrained → Zero-Shot on Lahore)

No Training Required for Model A

Model A requires no training at all. You simply load the eurosat_resnet50.pth checkpoint from Phase 1 exactly as saved and run it on your Lahore test patches directly. Do not update any weights. Do not use any Lahore training data. This condition measures the raw cross-regional transferability of EuroSAT-learned features to Lahore without any adaptation.

23. **Load:** Your eurosat_resnet50.pth checkpoint with the 13-channel input layer and 10-class output layer from Phase 1.
24. **Note on class mismatch:** EuroSAT has 10 classes; your Lahore schema may have 7. Map EuroSAT class predictions to your Lahore classes by grouping (e.g., EuroSAT Residential → Lahore Residential). Document your mapping table clearly in your notebook.
25. **Evaluate:** Run on the Lahore test set. Record OA, per-class F1, Kappa, and confusion matrix using your mapped class IDs.

Step 4.4 — Model B (EuroSAT Pretrained → Fine-tuned on Lahore)

26. **Load:** Your eurosat_resnet50.pth checkpoint.
27. **Modify:** Replace the final FC layer to match your Lahore class count.
28. **Train:** Fine-tune all layers on Lahore training set using identical hyperparameters to the Baseline (AdamW, LR 1×10^{-4}, cosine annealing, 30 epochs, batch 16, early stopping 5).
29. **Save:** Save best checkpoint as modelB_lahore.pth.
30. **Evaluate:** Run on the Lahore test set. Record OA, per-class F1, Kappa, and confusion matrix.

Step 4.5 — Compile Comparison Table

After all three evaluations, compile results into a single comparison table in a CSV file called results_comparison.csv with columns: model_name, overall_accuracy, kappa, macro_f1, and one column per class for per-class F1. This is the core results table for your paper.

> **Expected Ordering of Results**
> Model B should outperform Baseline, which should outperform Model A. If Model A outperforms the Baseline, this is an interesting finding — it would suggest ImageNet pretraining is actually harmful for this domain, which is a publishable result in itself. Do not adjust anything to force the expected ordering. Report what you actually find.

PHASE 5 Sample Efficiency Experiment

This experiment answers Research Question 3: how many Lahore-specific labeled samples does Model B need to achieve acceptable performance? This is one of the most practically useful findings for South Asian remote sensing practitioners.

PHASE 5 Sample Efficiency Experiment

Duration: 2 Weeks · Target: August 2026, Weeks 1–2

Step 5.1 — Define Sample Sizes

You will retrain Model B five separate times, each time using a progressively smaller subset of your Lahore training data. Use these exact sample sizes per class:

Run	Samples/Class	Approx. Total	Expected OA	Purpose
Run 1	500	~3,500	87–93%	Upper bound
Run 2	300	~2,100	84–90%	Near full
Run 3	200	~1,400	80–87%	Mid range
Run 4	100	~700	72–82%	Low data
Run 5	50	~350	60–74%	Minimum data

Step 5.2 — Execution Protocol

31. **Use fixed random seeds:** When subsampling, always set numpy.random.seed(42) before sampling. This ensures your subsets are reproducible and nested — the 100-sample subset is always a subset of the 300-sample subset, and so on.
32. **Always start from the EuroSAT checkpoint:** Each of the five runs must re-initialize from eurosat_resnet50.pth. Never continue from a previous run's checkpoint.
33. **Use the full test set for evaluation:** Evaluate every run on the same complete Lahore test set (30 samples per class). Never subsample the test set.
34. **Record metrics per run:** Save OA, Kappa, and macro F1 for each run to a CSV called sample_efficiency_results.csv.

Step 5.3 — Plot the Learning Curve

Create a line chart with training samples per class on the x-axis and Overall Accuracy on the y-axis. Draw a horizontal dashed red line at 85% OA — this is your acceptability threshold. The point where the curve first crosses above this line identifies your minimum required sample size. Include error bars if you run each condition multiple times with different random seeds (recommended but optional given time constraints).

Step 5.4 — Interpret the Finding

Your minimum sample threshold finding is a concrete, actionable recommendation for South Asian LULC practitioners. Write it as a clear statement: for example, 'Model B requires a minimum of X labeled samples per class to achieve 85% Overall Accuracy when fine-tuning from an EuroSAT-pretrained ResNet-50 checkpoint in the Lahore domain.' This goes directly into your paper's Discussion and Abstract.

PHASE 6 Domain Shift Analysis — t-SNE Visualization

This phase quantifies and visualizes the spectral and structural domain shift between EuroSAT's European imagery and your Lahore Sentinel-2 imagery. The t-SNE plots and per-class ΔF1 scores are among the most visually compelling outputs of your study.

PHASE 6 Domain Shift & t-SNE Analysis

Duration: 2 Weeks · Target: August 2026, Weeks 3–4

Step 6.1 — Extract Feature Vectors

Use your EuroSAT-pretrained ResNet-50 checkpoint (before any Lahore fine-tuning) as a frozen feature extractor. Remove the final classification layer and use the 2,048-dimensional output of the average pooling layer as your feature representation. Extract feature vectors for:

- **EuroSAT test patches:** Use approximately 500 patches total (50 per class) to keep t-SNE computation manageable.
- **Lahore test patches:** Use all your available test patches (30 per class minimum).
- **Label each vector:** with (1) its true class name and (2) its domain tag (EuroSAT or Lahore). You will use both for coloring the t-SNE plot.
- **Save:** as a numpy array features.npy and a matching labels CSV.

Step 6.2 — Run t-SNE

- **Combine:** Stack EuroSAT and Lahore feature matrices into a single array.
- **Run t-SNE:** Use scikit-learn's TSNE with perplexity=30, n_iter=1000, random_state=42, n_components=2.
- **Runtime:** Expect 10–20 minutes on your local CPU. This is normal — t-SNE is computationally expensive but parallelizes on multi-core CPUs.
- **Repeat after fine-tuning:** Extract features again using your Model B checkpoint (after Lahore fine-tuning) and run t-SNE a second time. The before/after comparison is your key visualization.

Step 6.3 — Create Publication-Quality t-SNE Plots

Create two side-by-side scatter plots: (1) Before fine-tuning — features from EuroSAT pretrained model. (2) After fine-tuning — features from Model B. Use consistent colors: one color per class, and distinguish EuroSAT vs. Lahore points using different marker shapes (circles for EuroSAT, triangles for Lahore). Well-separated clusters indicate classes that transfer well. Overlapping EuroSAT and Lahore clusters for the same class indicate successful transfer or fine-tuning alignment.

Step 6.4 — Compute Per-Class Domain Gap (ΔF1)

For each LULC class, compute the domain gap score as: ΔF1 = F1(EuroSAT test, class) − F1(Lahore zero-shot, class). A large positive ΔF1 means the class transfers poorly. A ΔF1 near zero means the class

features are geographically universal. Rank all classes by ΔF1 from largest to smallest. In your paper's Discussion section, explain the spectral or structural reason for each class's position in this ranking.

PHASE 7 Results, Figures & Paper Writing

This phase transforms your raw results into a complete, submission-ready manuscript. Allocate four full weeks — two for figures and results analysis, two for writing. Do not compress this phase.

PHASE 7 Results, Figures & Manuscript

Duration: 4 Weeks · Target: September 2026

Step 7.1 — Complete Figure List

Every figure must be saved at 300 DPI minimum in both PNG (for drafts) and PDF or EPS (for journal submission). Create all figures before writing — you will reference them constantly while drafting.

Fig.	Figure Title	Tool	What It Shows
1	**Study Area Map of Lahore District**	QGIS / ArcGIS Pro	Location, extent, and 2023 LULC distribution of the study area
2	**EuroSAT Sample Patches (10 Classes)**	Matplotlib	Visual reference of EuroSAT imagery classes for reader orientation
3	**Lahore Sample Patches (Your Classes)**	Matplotlib	Visual reference of Lahore patches matching EuroSAT classes
4	**ResNet-50 Architecture Diagram**	Draw.io / PowerPoint	Modified 13-channel input architecture schematic
5	**EuroSAT Training Loss and Accuracy Curves**	Matplotlib	Convergence behavior during Phase 1 training
6	**Confusion Matrix — Baseline**	Seaborn heatmap	Classification errors of ImageNet fine-tuned model
7	**Confusion Matrix — Model A (Zero-Shot)**	Seaborn heatmap	Zero-shot cross-regional errors by class
8	**Confusion Matrix — Model B**	Seaborn heatmap	Fine-tuned EuroSAT model classification errors
9	**Model Comparison Bar Chart**	Matplotlib	OA, Kappa, Macro F1 side by side for all three models
10	**Per-Class F1 Comparison**	Matplotlib grouped bars	F1 per class for all three models
11	**Domain Gap ΔF1 Chart**	Matplotlib horizontal bars	Class-level domain shift magnitude ranked
12	**t-SNE Before Fine-Tuning**	Matplotlib scatter	Feature space domain shift visualization
13	**t-SNE After Fine-Tuning**	Matplotlib scatter	Feature alignment after Model B adaptation

14	**Sample Efficiency Learning Curve**	Matplotlib line chart	OA vs. samples per class with 85% threshold line

Step 7.2 — Study Area Map (QGIS)

35. **Load layers:** Your Lahore Sentinel-2 composite, Lahore District boundary, Pakistan national boundary, and a basemap.
36. **Add cartographic elements:** North arrow, scale bar (show both km and miles), legend for LULC classes, inset locator map showing Lahore's position within Pakistan.
37. **Export:** From QGIS Layout Manager, export at 300 DPI as PNG and PDF.

Step 7.3 — Paper Structure (IMRAD Format)

Write your manuscript in this sequence. Writing the Results section first (before Discussion and Introduction) is recommended — it forces you to let the data drive your narrative rather than retrofitting results to a predetermined story.

Section	Key Content	Estimated Length
Abstract	OA values for all three models, domain shift finding, dataset contribution, minimum sample finding	250–300 words
1. Introduction	LULC importance, EuroSAT background, South Asian gap, Lahore context, study contributions	800–1,200 words
2. Related Work	EuroSAT & deep learning, transfer learning in RS, domain shift literature, Lahore LULC studies	800–1,000 words
3. Study Area	Geographic description, climate, land cover distribution, justification	400–600 words
4. Methodology	Dataset prep, model architecture, three experimental conditions, evaluation metrics	1,200–1,800 words
5. Results	Tables and figures with factual descriptions — no interpretation here	800–1,200 words
6. Discussion	Why Model B wins, class-level transfer explanation, t-SNE interpretation, practical implications	1,000–1,500 words
7. Conclusion	Key findings in 3–4 sentences, contributions, limitations, future work	300–400 words
References	All cited works in MDPI style	Typically 30–50 references

Step 7.4 — Dataset Publication on Zenodo

38. **Organize your dataset package:** lahore_patches/ folder (all 64×64 GeoTIFFs), ground_truth_final.csv, a README.md explaining the dataset, and eurosat_stats.json.

39. **Upload to Zenodo:** Create a new upload, fill in all metadata (title, authors, description, keywords, license = CC-BY 4.0).
40. **Publish and get DOI:** Once published, Zenodo assigns a permanent DOI. Add this DOI to your paper's Data Availability Statement and cite it in the manuscript.
41. **Publish code on GitHub:** Tag your GitHub repository with a v1.0 release. Include a requirements.txt and a clear README with instructions for reproducing each phase.

PHASE 8 Peer Feedback, Revision & Journal Submission

PHASE 8 Journal Submission

Duration: 3 Weeks · Target: October 2026

Step 8.1 — Internal Review Before Submission

- **Share with your supervisor:** Send the draft and all figures. Allow at least one week for feedback. Address all comments before submitting.
- **Self-checklist:** Every figure referenced in the text must be in the manuscript. Every table has a caption. Every reference in the text is in the reference list. Your data availability statement includes your Zenodo DOI.
- **Language check:** Read the entire manuscript out loud. This catches awkward phrasing that spell-checkers miss. Consider using Grammarly for grammar.

Step 8.2 — Format for MDPI Remote Sensing

42. **Download the template:** Go to mdpi.com/journal/remotesensing/instructions and download the LaTeX or Word author template.
43. **Format figures:** MDPI requires figures to be submitted as separate files (TIFF or PDF preferred) at 300 DPI. Do not embed them in the Word template during initial submission.
44. **Supplementary materials:** Additional confusion matrices, extended sample efficiency tables, and your full dataset description can go in a Supplementary Materials file.

Step 8.3 — Write the Cover Letter

The cover letter is addressed to the Editor-in-Chief of Remote Sensing (MDPI). It should be one page and cover: (1) the paper title and what it investigates, (2) why it fits the Remote Sensing journal scope specifically, (3) the key original contribution in two to three sentences, (4) a statement confirming the work is original and not under consideration elsewhere, (5) three to five suggested reviewer names with affiliations and email addresses — choose authors of papers you cited who work in transfer learning or LULC deep learning.

Step 8.4 — Submit via MDPI Online System

45. **Go to:** susy.mdpi.com and register or log in.
46. **Select journal:** Remote Sensing, Article Type: Article.
47. **Upload files:** Main manuscript, cover letter, all figures as separate files, supplementary materials.
48. **Fill metadata:** Title, abstract, keywords (use your proposal keywords exactly), author information, suggested reviewers.
49. **Submit:** After submission you will receive a manuscript ID. Average review time for Remote Sensing is 6–8 weeks.

Step 8.5 — Responding to Reviewers

- **Typical outcomes:** Major revision (most common for first submissions — do not be discouraged), minor revision, reject and resubmit, or accept.
- **Response strategy:** For every reviewer comment, write a point-by-point response. If you disagree with a comment, explain your reasoning politely and with evidence — you are not required to blindly accept all suggestions.
- **Timeline:** MDPI typically gives 10–14 days for minor revisions and 30 days for major revisions. Request an extension if you need one — editors almost always grant it.

MASTER Week-by-Week Timeline

Month	Week	Tasks	Key Deliverable
April	Week 1	Accounts, software installation, conda environment, folder structure	Dev environment verified, GitHub repo live
April	Week 2	EuroSAT download, upload to Drive, compute band statistics	eurosat_stats.json saved
April	Week 3	EuroSAT dataset loader, ResNet-50 13-channel modification	Loader returns correct tensor shapes
May	Week 1	EuroSAT training run on Colab T4 (50 epochs)	eurosat_resnet50.pth checkpoint saved
May	Week 2	EuroSAT evaluation, checkpoint download, GEE account setup	EuroSAT OA > 93% confirmed
May	Week 3	GEE Sentinel-2 composite for Lahore, export to Drive	lahore_sentinel2.tif downloaded
May	Week 4	Patch tiling script, metadata CSV generation	~4,000–8,000 patch files created
June	Week 1	QGIS setup, class schema definition, labeling begins	Class schema documented
June	Week 2	Ground truth labeling continues (target: 50+ per class)	~350 patches labeled
June	Week 3	Ground truth labeling continues, field verification trip	~700 patches labeled
June	Week 4	Labeling completion, field verification, label QC, train/test split	ground_truth_final.csv exported
July	Week 1	Lahore dataset loader, Baseline training on Lahore	baseline_lahore.pth saved
July	Week 2	Model A zero-shot evaluation on Lahore	Model A metrics recorded
July	Week 3	Model B fine-tuning on Lahore	modelB_lahore.pth saved
July	Week 4	All three models evaluated, comparison table compiled	results_comparison.csv complete
August	Week 1	Sample efficiency runs (500, 300, 200 samples/class)	3 runs complete
August	Week 2	Sample efficiency runs (100, 50 samples/class), learning curve plot	sample_efficiency_results.csv
August	Week 3	Feature extraction for t-SNE, t-SNE computation (before fine-tuning)	t-SNE plot 1 saved
August	Week 4	t-SNE after fine-tuning, ΔF1 domain gap analysis	t-SNE plot 2, domain gap table saved
September	Week 1	All 14 figures created and exported at 300 DPI	Complete figure set

September	**Week 2**	Results and Methods sections written	**2,000–3,000 words drafted**
September	**Week 3**	Introduction, Discussion, Conclusion written	**Full draft complete**
September	**Week 4**	Dataset upload to Zenodo, code release on GitHub	**DOI obtained**
October	**Week 1**	Supervisor review, revision based on feedback	**Revised manuscript**
October	**Week 2**	MDPI template formatting, cover letter written	**Submission package ready**
October	**Week 3**	Journal submission via MDPI SUSY system	**Manuscript ID received**

QUICK Quick Reference Card

Where to Run Each Task

Task	Platform	Reason
EuroSAT training (50 epochs)	**Google Colab — T4 GPU**	T4 is 4× faster than your Quadro P2000
Lahore Sentinel-2 acquisition	**Google Earth Engine (browser)**	GEE processes satellite data server-side
Patch tiling with Rasterio	**Local machine — CPU**	No GPU needed; fast on i7-8850H
Ground truth labeling	**Local machine — QGIS**	Requires visual inspection; no cloud equivalent
Baseline fine-tuning (30 epochs)	**Local machine — Quadro P2000**	Small dataset; takes only 1–2 hours locally
Model B fine-tuning (30 epochs)	**Local machine — Quadro P2000**	Same as above
Model A zero-shot evaluation	**Local machine**	Just loading a checkpoint and running inference
Sample efficiency runs (5×)	**Local or Colab**	Each run is short; local is fine
t-SNE computation	**Local machine — CPU**	10–20 min on 6-core i7; no GPU required
Figure generation	**Local machine — Matplotlib**	No GPU needed
Paper writing	**Local machine — Word / LaTeX**	Offline work

Critical Success Criteria — Never Skip These Checks

	Checkpoint	What to Verify
✓	**After Phase 1 training**	EuroSAT test Overall Accuracy ≥ 93%. If below, fix preprocessing before proceeding.
✓	**After Sentinel-2 export**	Visually inspect composite in QGIS. No large data gaps or cloud artifacts.
✓	**After patch tiling**	Load 5 random patches in Python. Confirm shape = (10, 64, 64) and no all-zero bands.
✓	**After ground truth**	Every class has ≥ 100 training and ≥ 30 test samples. Train/test split is fixed.
✓	**After all model training**	Model B OA > Baseline OA. If not, investigate — do not re-label to force this.
✓	**Before writing**	All figures saved at 300 DPI. Zenodo DOI obtained. GitHub repository tagged.

✓	**Before submission**	Cover letter written. MDPI template applied. Suggested reviewers identified.

Key File Inventory — Everything That Must Be Saved

File / Asset	Location	Description
eurosat_stats.json	models/	13-band mean and std for normalization
eurosat_resnet50.pth	models/ + Google Drive	Best EuroSAT checkpoint — most critical file
lahore_sentinel2.tif	data/lahore_sentinel2/	Full Lahore Sentinel-2 composite (2023)
lahore_patch_XXXX.tif	data/lahore_patches/	All 64×64 tiled patches (~4,000–8,000 files)
patch_metadata.csv	data/lahore_patches/	Patch ID, coordinates, lat/lon for each patch
ground_truth_final.csv	data/ground_truth/	All labels with class, split, confidence, source
baseline_lahore.pth	models/	Baseline model checkpoint
modelB_lahore.pth	models/	Model B best checkpoint
results_comparison.csv	results/	OA, Kappa, F1 for all three models
sample_efficiency_results.csv	results/	OA vs. sample size for 5 runs
features_before.npy	results/	2048-dim features for t-SNE before fine-tuning
features_after.npy	results/	2048-dim features for t-SNE after fine-tuning
Figures 1–14 (PNG + PDF)	figures/	All publication-ready figures at 300 DPI

Complete Analysis Execution Plan — Muhammad Shahid Usman — MS Data Science, Riphah International University — April 2026